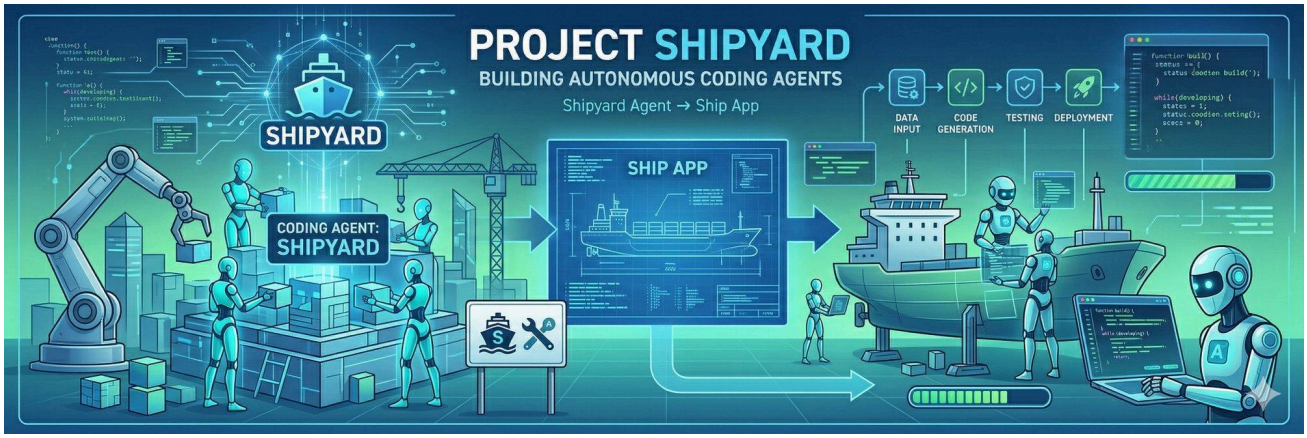

Shipyard

Building an Autonomous Coding Agent

Before You Start: Pre-Search (4 Hours)

Before writing any code, complete the Pre-Search methodology at the end of this document. This structured process uses AI to research real coding agent architectures, define your own approach, and lock architecture decisions before you are under time pressure. Your Pre-Search output becomes part of your final submission.

This week emphasizes careful system design over fast shipping. Pre-Search is where you decide what your agent does and how — not a box to check on the way to coding.

Background

Claude Code exists. Cursor exists. Both of them solve the same hard problem: an AI that can read a codebase, understand what needs to change, and make targeted edits without breaking everything around them.

The engineering is less obvious than it looks. Rewriting entire files is easy. Making a targeted change to one function in a 2,000-line file — without touching anything else, without losing context, without hallucinating imports — requires deliberate design. Keeping an agent running continuously, coordinating multiple agents in parallel, knowing what context to inject and when: these are real problems that different tools solve differently, and the decisions you make early determine whether your agent is useful or a liability.

Your job is to build your own version. Design and implement a coding agent from scratch, then use it to rebuild the Ship app. The rebuild is your integration test — it proves your agent works under real conditions. When it's done, produce a rigorous comparative analysis: how does the version your agent built compare to the original? Where did it fall short? Where did it surprise you? What would you change?

There is no prescribed architecture. There is a defensible one. Finding it, justifying it, and building it is the assignment.

Project Overview

One-week sprint with three deadlines:

Checkpoint	Deadline	Focus
Pre-Search	4 hours after assignment	Open source research, architecture design, file editing strategy locked
MVP	Tuesday, 11:59 PM	Running agent, surgical edits working, context injection functional, tracing enabled
Early Submission	Thursday, 11:59 PM	Ship rebuild complete, comparative analysis drafted, multi-agent coordination working
Final Submission	Sunday, 11:59 PM	All deliverables submitted, documentation complete, deployed

Project completion is required to move onto the next week.

MVP Requirements (36 Hours)

Hard gate. All items required to pass:

- ☐ Agent runs in a persistent loop and accepts new instructions without restarting
- ☐ Surgical file editing implemented — agent makes targeted changes without rewriting entire files
- ☐ Context injection functional — agent accepts injected context at runtime and uses it in generation
- ☐ Tracing enabled — at least two shared trace links submitted showing different execution paths
- ☐ PRESEARCH.md submitted with research notes and all architecture artifacts
- ☐ Accessible via GitHub — runs locally, no deployment required at this stage
- ☐ CODEAGENT.md submitted with Agent Architecture and File Editing Strategy sections complete

A focused agent that surgical edits and runs continuously beats a feature-rich agent that rewrites files and crashes.

Core Agent

Agent Requirements

Requirement	What It Means
Continuous operation	The agent runs in a persistent loop and accepts new instructions without restarting. Fire-and-forget invocations do not count.
Surgical file editing	The agent makes targeted changes to specific lines or blocks without rewriting entire files. See the File Editing section below.
Multi-agent coordination	The system can spawn and coordinate multiple agents working in parallel or in sequence and merge their outputs correctly.
Context injection	External context — a spec, a schema, a previous output, a test result — can be injected at runtime and used in the agent's next action.

File Editing

This requirement gets its own section because it is the most commonly skipped or faked. An agent that rewrites entire files on every change is not a coding agent — it is a code generator with a file system wrapper. It will be evaluated accordingly.

Strategies to research and choose from:

- Unified diff (git-style patches): generate diffs, apply with standard patch tooling. Precise and auditable, but requires the LLM to produce well-formed diffs consistently.
- Line-range replacement: identify start and end line numbers, replace only that range. Simple to implement, fragile if line numbers drift.
- AST-based editing: parse the file to an abstract syntax tree, modify the target node, serialize back. Maximally precise, but language-specific and complex to implement.
- Anchor-based replacement: use unique string anchors within the file to locate and replace specific blocks. More robust than line numbers, less complex than AST.

Pick one strategy, justify your choice in PRESEARCH.md, and implement it correctly. Switching strategies mid-week because you did not think it through is a planning failure.

Multi-Agent Coordination

Your system must support spawning and coordinating at least two agents. Document your orchestration model: how agents communicate, how outputs are merged, and how conflicts are resolved. LangGraph is recommended; any other framework is permitted if you produce equivalent run traces.

Ship App Rebuild

Once your agent is functional, use it to rebuild the Ship app — all current features — from scratch. You are not building a clone by hand. You are directing your agent to build it, then documenting and analyzing what it produces.

The rebuild serves two purposes. First, it is your integration test: if your agent cannot complete a real build task, your agent is not done. Second, it generates the data for your comparative analysis. Every shortcoming your agent has will surface here — that is the point.

You are expected to intervene when the agent gets stuck, but document every intervention. Interventions are data, not failures.

Comparative Analysis

After the rebuild, produce a structured written analysis comparing the agent-built version of Ship against the original. All seven sections are required. This is the most heavily weighted deliverable — honest, specific analysis of a flawed agent scores higher than vague praise of a polished one.

Section	What to Cover
Executive Summary	One paragraph: what you built and how the rebuild went overall
Architectural Comparison	How does the agent-built version differ structurally from the original? What choices did your agent make that a human developer would not have?
Performance Benchmarks	Specific, measurable comparisons — code complexity, test coverage, load time, lines of code, or whatever is meaningful for Ship
Shortcomings	Where did your agent fail, produce incorrect output, or require intervention? List every intervention from your rebuild log and what it reveals about your agent's limitations.
Advances	Where did your agent outperform or move faster than manual development?
Trade-off Analysis	For each major architecture decision in your agent, was it the right call? What would you change?
If You Built It Again	What would be different about your agent's architecture, file editing strategy, or context management?

Vague analysis will be penalized. Specific claims with evidence from the rebuild log are required.

Observability

Every agent run must be traceable. You must be able to answer after any run: what did the agent do, in what order, with what inputs, and what did it produce at each step? LangSmith tracing is recommended if you are using LangGraph — it traces automatically once environment variables are set. If you are using a different framework, you are responsible for producing equivalent traces.

AI-First Development Requirements

This week emphasizes learning AI-first development workflows. You must document your process.

AI Development Log (Required)

Submit a 1-page document covering:

Section	Content
Tools & Workflow	Which AI coding tools you used and how you integrated them
Effective Prompts	3–5 prompts that worked well — include the actual prompts, not descriptions of them
Code Analysis	Rough percentage of AI-generated vs. hand-written code
Strengths & Limitations	Where the tools excelled and where they fell short on this specific project
Key Learnings	What you would do differently when using coding agents on your next project

AI Cost Analysis (Required)

Understanding AI costs is critical for production applications. Submit a cost analysis covering:

Development and Testing Costs

Track and report your actual spend during development:

- Claude API costs (input and output token breakdown)
- Number of agent invocations during development
- Total development spend

Production Cost Projections

Estimate monthly costs at different user scales:

100 Users	1,000 Users	10,000 Users
$___/month	$___/month	$___/month

Include assumptions: average agent invocations per user per day, average tokens per invocation (input/output breakdown), cost per invocation.

Open Source Research

Study at least two of the following coding agents before writing code. Read the source, understand the architecture, and document what you learn in PRESEARCH.md. These agents were selected because they each make different, defensible design decisions.

Agent	Why It's Worth Studying	Where to Find It
OpenCode	Modern TypeScript coding agent. Strong reference for tool design, context window management, and agent loop structure.	github.com/opencode-ai/opencode
LangChain Open Engineer	Multi-agent orchestration patterns, memory management, tool composition across agents.	github.com/langchain-ai/open-engineer
Claude Code (docs)	Architecture overview, permission model, sub-agent coordination, human-in-the-loop design.	docs.anthropic.com/claude-code

Technical Stack

Possible Paths

Layer	Technology
Agent Framework	LangGraph (recommended), LangChain, or custom loop
LLM	Claude (Anthropic SDK required)
Observability	LangSmith, Langfuse, or custom structured logging
Backend	Python/FastAPI or Node.js/Express
Deployment	Runs locally for MVP; deployed for Final Submission

Build Strategy

Priority order:

1. Persistent loop — agent accepts instructions and runs continuously
2. Basic tool calls — read_file and edit_file working end-to-end
3. Surgical file editing — implement and verify your chosen strategy
4. Context injection — external context accepted and used
5. Multi-agent coordination — spawn two agents, merge outputs
6. Ship rebuild — direct your agent at Ship, document everything
7. Comparative analysis — write the full seven-section report

Critical Guidance

- Get surgical file editing working completely before moving to multi-agent
- Test your edit strategy against files of varying size — behavior often breaks above 200 lines
- Document every intervention during the Ship rebuild — this is your analysis data
- Add tracing early — you need visibility to debug agent behavior
- Do not mock responses at any stage — run against real code

Submission Requirements

Deliverable	Requirements
GitHub Repository	Setup guide, architecture overview; another engineer can clone and run without asking you questions
Demo Video (3–5 min)	Show the agent making a surgical edit, completing a multi-agent task, and at least one example from the Ship rebuild
PRESEARCH.md	Completed pre-search checklist from the appendix below
CODEAGENT.md	All sections complete — see appendix below
AI Development Log	1-page breakdown using the template above
AI Cost Analysis	Dev spend + projections for 100 / 1K / 10K users
Deployed Application	Agent and agent-built Ship app both publicly accessible
Social Post	Share on X or LinkedIn: description, features, demo or screenshots, tag @GauntletAI

Project completion is required to move onto the next week.

Appendix: Pre-Search Checklist

Complete this before writing any code. Save your AI conversation as a reference document. The goal is to make informed decisions about your agent's architecture before you are under time pressure. You do not have to be right — you have to show your thought process.

Phase 1: Open Source Research

1. For each agent you studied:
    - Which agent, and which parts of the source code you read
    - How it handles file editing — mechanism, tradeoffs, failure modes
    - How it manages context across turns
    - How it handles failed tool calls and unexpected output
    - What you would take from it
    - What you would do differently, and why

2. Which file editing strategy are you adopting and why? What are its failure modes and how will you handle them?

Phase 2: Architecture Design

3. System diagram — full flow from user instruction through agent loop to output, including at least one error branch (Mermaid or image)

4. File editing strategy — describe the mechanism step by step. What does the agent do when it gets the location wrong?

5. Multi-agent design — orchestration model, how agents communicate, how outputs are merged

6. Context injection spec — what types of context, in what format, at what point in the loop

7. Context injection spec — what types of context, in what format, at what point in the loop

8. Any additional tools your agent will need — list them with a one-sentence description of each

Phase 3: Stack and Operations

9. What framework are you using for the agent loop and multi-agent coordination, and why?

10. Where does your persistent loop run? How is it kept alive between instructions?

11. What is your token budget per invocation? Where are the cost cliffs?

12. What does your agent do when it makes a bad edit? How does it detect and recover?

13. What gets logged? Describe what a complete run trace looks like for a typical edit.

Appendix: CODEAGENT.md

Submission template — fill in each section as you build. Do not write this at the end.

Section	Due
Agent Architecture	MVP
File Editing Strategy	MVP
Multi-Agent Design	MVP
Trace Links	MVP
Architecture Decisions	Final Submission
Ship Rebuild Log	Final Submission
Comparative Analysis	Final Submission
Cost Analysis	Final Submission

Agent Architecture (MVP)

Diagram or written description of your full agent architecture — loop design, tool calls, state management, entry and exit conditions for normal runs and error branches.

File Editing Strategy (MVP)

Describe exactly how your agent makes surgical edits, step by step. What is the mechanism? How does it locate the correct block? What happens when it gets the location wrong?

Multi-Agent Design (MVP)

Orchestration model, how agents communicate, how parallel outputs are merged. Include a diagram if it helps.

Trace Links (MVP)

- Trace 1 (normal run):
- Trace 2 (different execution path — error, branching condition, or different task type):

Architecture Decisions (Final Submission)

Key architecture decisions, what you considered, and why you made the call you did.

Ship Rebuild Log (Final Submission)

Running log of the rebuild. For every human intervention: what broke or got stuck, what you did, and what it reveals about your agent's limitations.

Comparative Analysis (Final Submission)

All seven sections required — see main PRD for structure. Specific claims with evidence. No vague summaries.

Cost Analysis (Final Submission)

Item	Amount
Claude API — input tokens	
Claude API — output tokens	
Total invocations during development	
Total development spend	

100 Users	1,000 Users	10,000 Users
$___/month	$___/month	$___/month

Assumptions:

- Average agent invocations per user per day:
- Average tokens per invocation (input / output):
- Cost per invocation: